Paarth Maisuria

ITAPA

Assignment

Due Date: 01 July

Eduvos

EDUV5606518



Eduvos

Your Education. Your Future.

Deliverables 1, 2, 3, 4:

Code:

```
import csv
import tkinter as tk    # Built the GUI with Tkinter
from tkinter import ttk
from tkinter import messagebox
import threading
import time
from datetime import datetime
from pathlib import Path


# Path to the CSV file where student records are stored
students_path = Path("module_assessment_grades 1 (2).csv")


# Used logging to record actions in the system and saved to a textfile - Used for multi-threading
def log_student(student_no, student_name, action):
    with open("student_log.txt", "a") as file:
        now = str(datetime.now().replace(microsecond = 0))
        file.write(student_no + " - " + student_name + " " + action + " at " + now + "\n") #Maybe add commas instead of +


# Read student data from the CSV file
def read_file():
    students = []
    header = []
    try:
        with open(students_path, "r", encoding = "utf-8") as file:
         reader = csv.reader(file)
         try:
          # Read the header row
          header = next(reader)
         except StopIteration:
```

```
            return students, header
        # Read the student rows
        students = list(reader)
        for row in reader:
         students.append(row)
    # Error handling for file errors if not found
    except FileNotFoundError:
        messagebox.showerror("Could not read the file because it does not exist.")
    return students, header

# Save student data to the CSV file
def save_student(students):
    # First writes the header and then the student records from the CSV file
    try:
        with open(students_path, "w", encoding = "utf-8", newline = "") as file:
            writer = csv.writer(file)
            writer.writerow(["Student No", "Name", "Surname", "Module", "Quiz(10%)", "Project(20%)",
"Final Exam(50%)", "Practical(20%)", "Overall Grade"])
            writer.writerows(students)
    except Exception as e:
        messagebox.showerror("Error:", e)

# Window for Capture Student Marks
def capture_marks():
    # Window for user to input marks
    window = tk.Toplevel()
    window.title("Capture Student Marks")

    # Input fields for student details and marks
    tk.Label(window, text = "Student No:").pack()
    entry_no = tk.Entry(window)
    entry_no.pack()
```

```
tk.Label(window, text = "Name:").pack()
entry_name = tk.Entry(window)
entry_name.pack()

tk.Label(window, text = "Surname:").pack()
entry_surname = tk.Entry(window)
entry_surname.pack()

tk.Label(window, text = "Module:").pack()
entry_module = tk.Entry(window)
entry_module.pack()

tk.Label(window, text = "Quiz (10%):").pack()
entry_quiz = tk.Entry(window)
entry_quiz.pack()

tk.Label(window, text = "Project (20%):").pack()
entry_project = tk.Entry(window)
entry_project.pack()

tk.Label(window, text = "Final Exam (50%):").pack()
entry_exam = tk.Entry(window)
entry_exam.pack()

tk.Label(window, text = "Practical (20%):").pack()
entry_practical = tk.Entry(window)
entry_practical.pack()

# Save the captured marks
def save_marks():
```

```
    # Check for required text fields
    if entry_no.get().strip() == "" or entry_name.get().strip() == "" or entry_surname.get().strip() == "" or entry_module.get().strip() == "":
        messagebox.showerror("Error", "Student No, Name, Surname, and or Module are required to be populated")
        return

    # Check for required numeric fields
    if entry_quiz.get().strip() == "" or entry_project.get().strip() == "" or entry_exam.get().strip() == "" or entry_practical.get().strip() == "":
        messagebox.showerror("Error", "All marks are required to be populated")
        return

    # Remove any white spaces before storing and doing any calculations
    try:
        overall = calculate_grade(entry_quiz.get().strip(), entry_project.get().strip(), entry_exam.get().strip(), entry_practical.get().strip())
    except:
        messagebox.showerror("Error", "Marks should only contain numbers")
        return

    # Checking for marks in range from 0 to 100
    if overall == "Invalid mark" or overall == "Mark out of range":
        messagebox.showerror("Error", "Marks must be between 0 and 100")
        return

    # Save student record without any white spaces
    student = [entry_no.get().strip(), entry_name.get().strip(), entry_surname.get().strip(), entry_module.get().strip(),
               entry_quiz.get().strip(), entry_project.get().strip(), entry_exam.get().strip(), entry_practical.get().strip(), overall]

    students, header = read_file()
    students.append(student)
```

```
        save_student(students)

        # Logging action for saved marks in the background - used for multi-threading
        threading.Thread(target = log_student, args = (entry_no.get().strip(), entry_name.get().strip(), "saved")).start()

        tk.Label(window, text = f"Saved. Overall Grade: {overall}").pack()

    # Button to save marks using Tkinter
    tk.Button(window, text = "Save", command = save_marks).pack()

# Calculating the grade
def calculate_grade(quiz, project, exam, practical):
    # Converts the inputted marks to float
    try:
        quiz = float(quiz)
        project = float(project)
        exam = float(exam)
        practical = float(practical)
    except ValueError:
        return ("Invalid mark")

    # Checks if the marks are in the range between 0 and 100
    if quiz < 0 or quiz > 100:
        return ("Mark out of range")
    if project < 0 or project > 100:
        return ("Mark out of range")
    if exam < 0 or exam > 100:
        return ("Mark out of range")
    if practical < 0 or practical > 100:
        return ("Mark out of range")
```

```
    # Calculation from given weightings in question
    total = (quiz * 0.1) + (project * 0.2) + (exam * 0.5) + (practical * 0.2)
    return total

# Window for View / Delete Records
def view_delete():
    # Window for user to view or delete records
    window = tk.Toplevel()
    window.title("View/Delete Records")

    # To read student records from the CSV file
    students, header = read_file()

    # Formatting and displaying records into listbox
    listbox = tk.Listbox(window, width = 120)
    listbox.pack()

    # Format and display header row
    if header:
        listbox.insert(tk.END,
f"{header[0]} | {header[1]} | {header[2]} | {header[3]} | {header[4]} | {header[5]} | {header[6]}
 | {header[7]} | {header[8]}")

    # Format and display student records from the CSV file
    for s in students:
       listbox.insert(tk.END, f"{s[0]}   | {s[1]}
| {s[2]} | {s[3]} | {s[4]} | {s[5]} | {s[6]} | {s[7]} | {s[8]}")

    # Delete student from the CSV file
    def delete_student():
        choice = listbox.curselection()
        if choice:
            index = choice[0]
```

```
            if index == 0:
                messagebox.showerror("Error", "Cannot delete header")
                return
            else:
                selected_student = students[index-1]
                student_no = selected_student[0]
                student_name = selected_student[1]
                students.pop(index-1)
                save_student(students)
                listbox.delete(index)
                # Logging action for deleted marks in the background - used for multi-threading
                threading.Thread(target = log_student, args = (student_no, student_name,
"deleted")).start()
            messagebox.showinfo("Success", "Student deleted")
        else:
            messagebox.showerror("Error", "Please select a student first")

    tk.Button(window, text = "Delete", command = delete_student).pack()

# Window for Search / Update Records
def search_update():
    # Window for user to search or update records
    window = tk.Toplevel()
    window.title("Search & Update Marks")

    tk.Label(window, text = "Search by Student No:").pack()
    entry_number = tk.Entry(window)
    entry_number.pack()

    listbox = tk.Listbox(window, width = 120)
    listbox.pack()
```

```
    # Dropdown box to select which marks to update
    tk.Label(window, text = "Select Assessment").pack()
    dropdown = ttk.Combobox(window, values = ["Quiz(10%)", "Project(20%)", "Final Exam(50%)",
"Practical(20%)", "Overall Grade"])
    dropdown.pack()

    tk.Label(window, text = "New Mark").pack()
    entry_mark = tk.Entry(window)
    entry_mark.pack()

    # Search students from the CSV file and display in the listbox
    def search():
        students, header = read_file()
        listbox.delete(0, tk.END)
        number = entry_number.get().strip()
        if number == "":
            messagebox.showerror("Error", "Student number is required to be populated")
            return
        found = False
        for s in students:
            if s[0] == number:
                listbox.insert(tk.END,
f"{header[0]} | {header[1]} | {header[2]} | {header[3]} | {header[4]} | {header[5]} | {header[6]}
 | {header[7]} | {header[8]}")
                listbox.insert(tk.END, f"{s[0]}   | {s[1]}
| {s[2]} | {s[3]} | {s[4]} | {s[5]} | {s[6]} | {s[7]} | {s[8]}")
                found = True
                break
        if not found:
            messagebox.showerror("Error", "Student not found")

    # Update student marks
    def update_student():
```

```
    students, header = read_file()
    number = entry_number.get().strip()
    new_mark = entry_mark.get().strip()
    assessment = dropdown.get().strip()

    # Checks if student number or other inputs are not empty
    if number == "":
        messagebox.showerror("Error", "Student number is required to be populated")
        return
    if new_mark == "":
        messagebox.showerror("Error", "New mark is required to be populated")
        return
    if assessment != "Overall Grade":
        try:
            float(new_mark)
        except:
            messagebox.showerror("Error", "Mark must be a number and please do not use '%'
sign")
            return

    # Find student and update marks
    for s in students:
        if s[0] == number:
            if assessment == "Quiz(10%)":
                s[4] = new_mark
            elif assessment == "Project(20%)":
                s[5] = new_mark
            elif assessment == "Final Exam(50%)":
                s[6] = new_mark
            elif assessment == "Practical(20%)":
                s[7] = new_mark
            elif assessment == "Overall Grade":
```

```
            s[8] = new_mark

    if assessment != "Overall Grade":
        try:
            quiz = float(s[4])
            project = float(s[5])
            exam = float(s[6])
            practical = float(s[7])

            # Checks if the marks are in the range between 0 and 100
            if quiz < 0 or quiz > 100:
                messagebox.showerror("Error", "Marks out of range")
                return
            if project < 0 or project > 100:
                messagebox.showerror("Error", "Marks out of range")
                return
            if exam < 0 or exam > 100:
                messagebox.showerror("Error", "Marks out of range")
                return
            if practical < 0 or practical > 100:
                messagebox.showerror("Error", "Marks out of range")
                return

            overall = (quiz * 0.1) + (project * 0.2) + (exam * 0.5) + (practical * 0.2)
            s[8] = overall
        except:
            s[8] = "Invalid"

    # Save updated records to the CSV file
    try:
        with open(students_path, "w", encoding = "utf-8", newline = "") as file:
            writer = csv.writer(file)
```

```
                writer.writerow(header)
                writer.writerows(students)

            # Logging action for updated marks in the background - used for multi-threading
            student_no = s[0]
            student_name = s[1]
            threading.Thread(target = log_student, args = (student_no, student_name,
"updated")).start()

            messagebox.showinfo("Success", assessment + " updated successfully")
            return
        except Exception as e:
            messagebox.showerror("Error:", e)

    messagebox.showerror("Student not found")

tk.Button(window, text = "Search", command = search).pack()
tk.Button(window, text = "Update Mark", command = update_student).pack()

# Main window
menu = tk.Tk()
menu.title("Eduvos Student Grade Calculator")

# Display text on the main form as titles
info1_text = ("Eduvos Student Grade Calculator")
info2_text = ("(2026 Final Project 2)")
info3_text = ("Manage Student Marks Easily")
tk.Label(menu, text = info1_text, font = ("Arial", 16), fg = "navy").pack(pady = 10)
tk.Label(menu, text = info2_text, font = ("Arial", 16), fg = "navy").pack(pady = 10)
tk.Label(menu, text = info3_text, font = ("Arial", 12), fg = "grey").pack(pady = 10)

# Display buttons on the main form for each function
```

```
# pady - formatting of button size up and down

# padx - formatting of button size left and right

tk.Button(menu, text = "Capture Student Marks", bg = "green", fg = "black", command =
capture_marks).pack(pady = 20, padx = 30)


tk.Button(menu, text = "View / Delete Records", bg = "lightblue", fg = "black", command =
view_delete).pack(pady = 20, padx = 30)

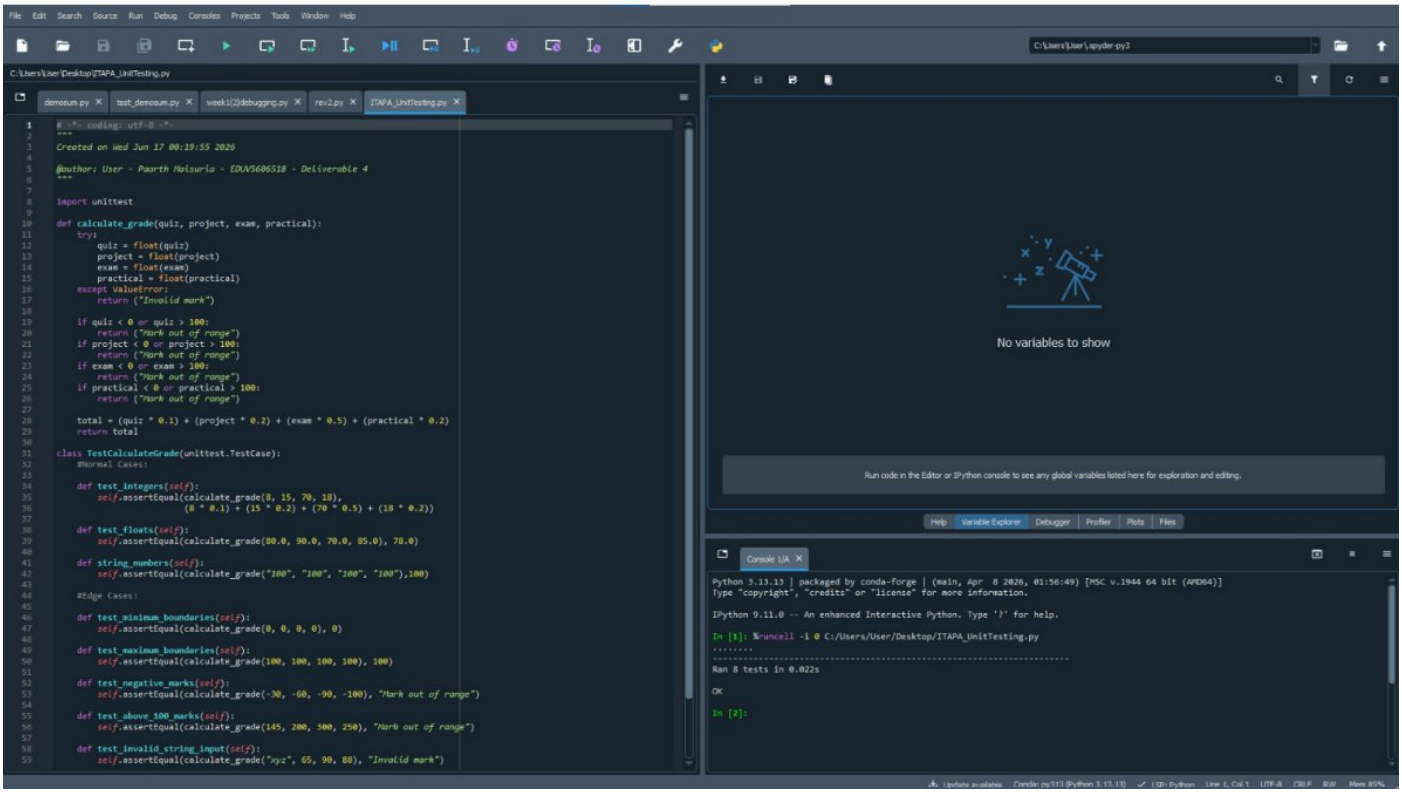

tk.Button(menu, text = "Search / Update Records", bg = "orange", fg = "black", command =
search_update).pack(pady = 20, padx = 30)

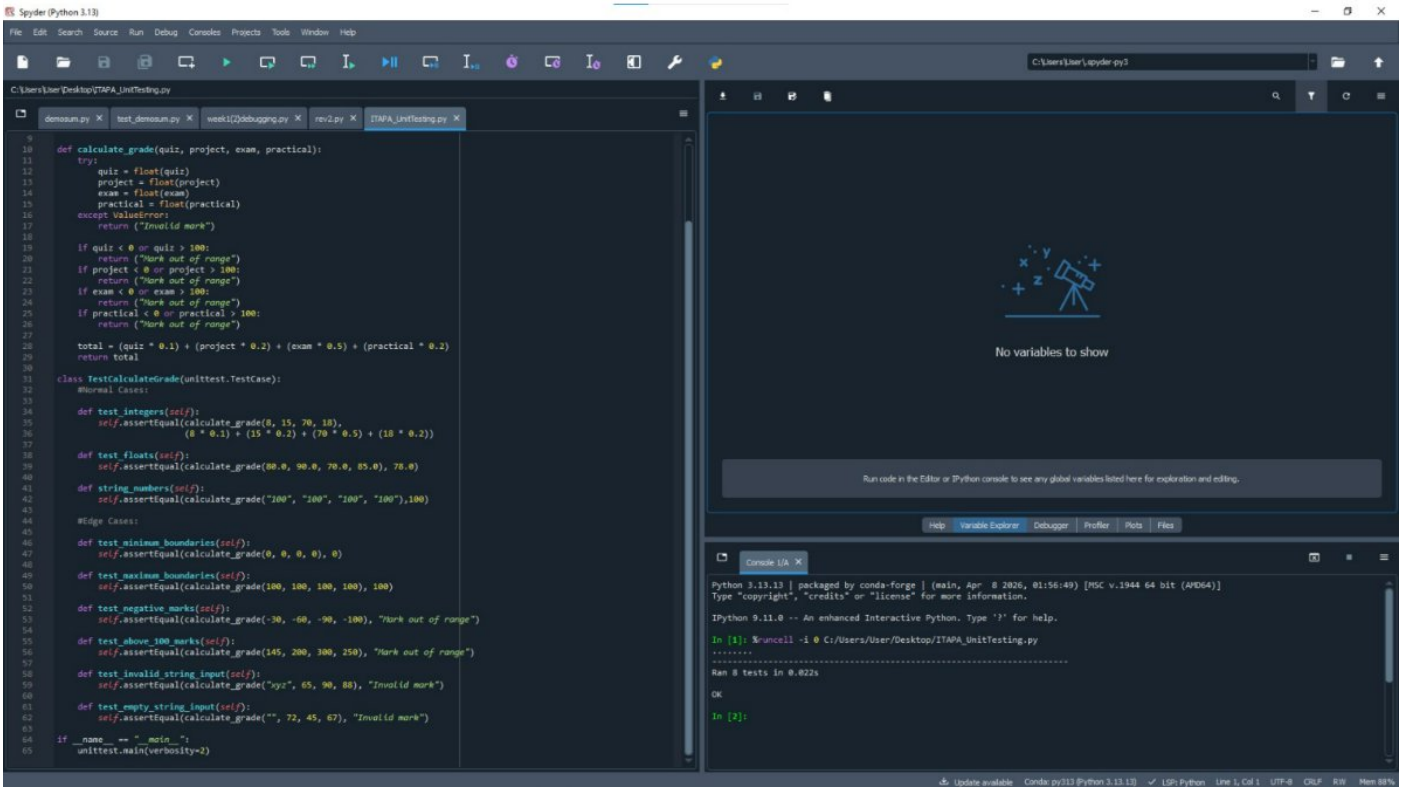

tk.Button(menu, text = "Close Application", bg = "red", fg = "black", command =
menu.destroy).pack(pady = 20, padx = 30)

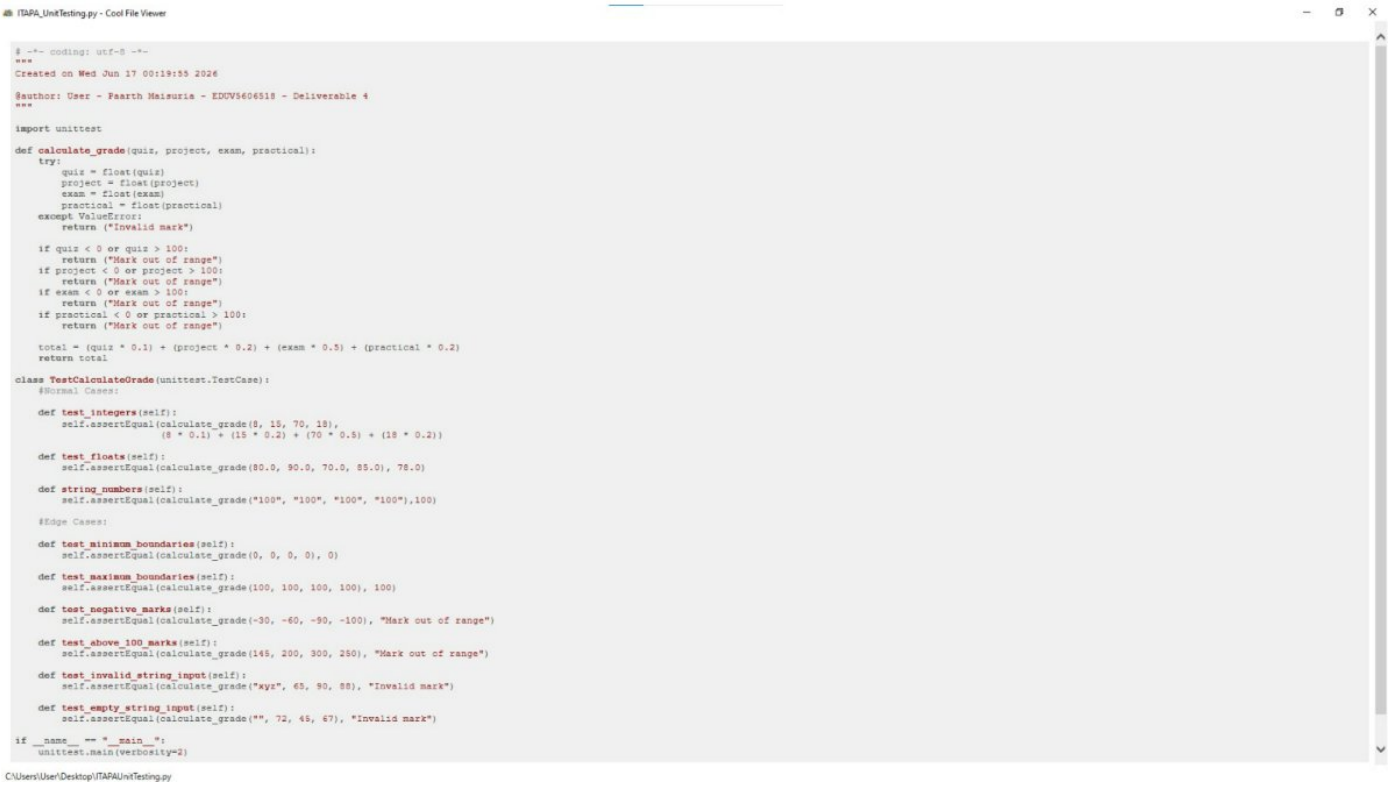

# Call the Tkinter function

menu.mainloop()
```

Unit Testing:


File Edit Search Source Run Debug Consoles Projects Tools Window Help
C:\Users\User\Desktop\ITAPA_UnitTesting.py
demosum.py
test_demosum.py
week1(2)debugging.py
rev2.py
ITAPA_UnitTesting.py
-*- coding: utf-8 -*-
"""
Created on Wed Jun 17 00:19:55 2026
@author: User - Paarth Naisuria - EDUV5606518 - Deliverable 4
"""
import unittest
def calculate_grade(quiz, project, exam, practical):
try:
quiz = float(quiz)
project = float(project)
exam = float(exam)
practical = float(practical)
except ValueError:
return ("Invalid mark")
if quiz < 0 or quiz > 100:
return ("Mark out of range")
if project < 0 or project > 100:
return ("Mark out of range")
if exam < 0 or exam > 100:
return ("Mark out of range")
if practical < 0 or practical > 100:
return ("Mark out of range")
total = (quiz * 0.1) + (project * 0.2) + (exam * 0.5) + (practical * 0.2)
return total
class TestCalculateGrade(unittest.TestCase):
#Normal Cases:
def test_integers(self):
self.assertEqual(calculate_grade(8, 15, 70, 18),
(8 * 0.1) + (15 * 0.2) + (70 * 0.5) + (18 * 0.2))
def test_floats(self):
self.assertEqual(calculate_grade(80.0, 90.0, 70.0, 85.0), 78.0)
def string_numbers(self):
self.assertEqual(calculate_grade("100", "100", "100", "100"),100)
#Edge Cases:
def test_minimum_boundaries(self):
self.assertEqual(calculate_grade(0, 0, 0, 0), 0)
def test_maximum_boundaries(self):
self.assertEqual(calculate_grade(100, 100, 100, 100), 100)
def test_negative_marks(self):
self.assertEqual(calculate_grade(-30, -60, -90, -100), "Mark out of range")
def test_above_100_marks(self):
self.assertEqual(calculate_grade(145, 200, 300, 250), "Mark out of range")
def test_invalid_string_input(self):
self.assertEqual(calculate_grade("xyz", 65, 90, 80), "Invalid mark")
No variables to show
Run code in the Editor or IPython console to see any global variables listed here for exploration and editing.
Help
Variable Explorer
Debugger
Profiler
Plots
Files
Console 1/A
Python 3.13.13 | packaged by conda-forge | (main, Apr 8 2026, 01:56:49) [MSC v.1944 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.
IPython 9.11.0 -- An enhanced Interactive Python. Type '?' for help.
In [1]: %runcell -i 0 C:/Users/User/Desktop/ITAPA_UnitTesting.py
Ran 8 tests in 0.022s
OK
In [2]:


Spyder (Python 3.13)
File Edit Search Source Run Debug Consoles Projects Tools Window Help
C:\Users\User\Desktop\ITAPA_UnitTesting.py
demosum.py
test_demosum.py
week1(2)debugging.py
rev2.py
ITAPA_UnitTesting.py
def calculate_grade(quiz, project, exam, practical):
try:
quiz = float(quiz)
project = float(project)
exam = float(exam)
practical = float(practical)
except ValueError:
return ("Invalid mark")
if quiz < 0 or quiz > 100:
return ("Mark out of range")
if project < 0 or project > 100:
return ("Mark out of range")
if exam < 0 or exam > 100:
return ("Mark out of range")
if practical < 0 or practical > 100:
return ("Mark out of range")
total = (quiz * 0.1) + (project * 0.2) + (exam * 0.5) + (practical * 0.2)
return total
class TestCalculateGrade(unittest.TestCase):
#Normal Cases:
def test_integers(self):
self.assertEqual(calculate_grade(8, 15, 70, 18),
(8 * 0.1) + (15 * 0.2) + (70 * 0.5) + (18 * 0.2))
def test_floats(self):
self.assertEqual(calculate_grade(80.0, 90.0, 70.0, 85.0), 78.0)
def string_numbers(self):
self.assertEqual(calculate_grade("100", "100", "100", "100"),100)
#Edge Cases:
def test_minimum_boundaries(self):
self.assertEqual(calculate_grade(0, 0, 0, 0), 0)
def test_maximum_boundaries(self):
self.assertEqual(calculate_grade(100, 100, 100, 100), 100)
def test_negative_marks(self):
self.assertEqual(calculate_grade(-30, -60, -90, -100), "Mark out of range")
def test_above_100_marks(self):
self.assertEqual(calculate_grade(145, 200, 300, 250), "Mark out of range")
def test_invalid_string_input(self):
self.assertEqual(calculate_grade("xyz", 65, 90, 80), "Invalid mark")
def test_empty_string_input(self):
self.assertEqual(calculate_grade("", 72, 45, 67), "Invalid mark")
if __name__ == "__main__":
unittest.main(verbosity=2)
No variables to show
Run code in the Editor or IPython console to see any global variables listed here for exploration and editing.
Help
Variable Explorer
Debugger
Profiler
Plots
Files
Console 1/A
Python 3.13.13 | packaged by conda-forge | (main, Apr 8 2026, 01:56:49) [MSC v.1944 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.
IPython 9.11.0 -- An enhanced Interactive Python. Type '?' for help.
In [1]: %runcell -i 0 C:/Users/User/Desktop/ITAPA_UnitTesting.py
Ran 8 tests in 0.022s
OK
In [2]:



Unit testing code:

```
# -- coding: utf-8 --
"""
Created on Wed Jun 17 00:19:55 2026

@author: User - Paarth Maisuria - EDUV5606518 - Deliverable 4
"""

import unittest

def calculate_grade(quiz, project, exam, practical):
    try:
        quiz = float(quiz)
        project = float(project)
        exam = float(exam)
        practical = float(practical)
    except ValueError:
        return ("Invalid mark")
```

```
    if quiz < 0 or quiz > 100:
        return ("Mark out of range")
    if project < 0 or project > 100:
        return ("Mark out of range")
    if exam < 0 or exam > 100:
        return ("Mark out of range")
    if practical < 0 or practical > 100:
        return ("Mark out of range")

    total = (quiz * 0.1) + (project * 0.2) + (exam * 0.5) + (practical * 0.2)
    return total

class TestCalculateGrade(unittest.TestCase):
    #Normal Cases:

    def test_integers(self):
        self.assertEqual(calculate_grade(8, 15, 70, 18),
                         (8 * 0.1) + (15 * 0.2) + (70 * 0.5) + (18 * 0.2))

    def test_floats(self):
        self.assertEqual(calculate_grade(80.0, 90.0, 70.0, 85.0), 78.0)

    def string_numbers(self):
        self.assertEqual(calculate_grade("100", "100", "100", "100"),100)

    #Edge Cases:

    def test_minimum_boundaries(self):
        self.assertEqual(calculate_grade(0, 0, 0, 0), 0)

    def test_maximum_boundaries(self):
        self.assertEqual(calculate_grade(100, 100, 100, 100), 100)
```

```
    def test_negative_marks(self):
        self.assertEqual(calculate_grade(-30, -60, -90, -100), "Mark out of range")

    def test_above_100_marks(self):
        self.assertEqual(calculate_grade(145, 200, 300, 250), "Mark out of range")

    def test_invalid_string_input(self):
        self.assertEqual(calculate_grade("xyz", 65, 90, 88), "Invalid mark")

    def test_empty_string_input(self):
        self.assertEqual(calculate_grade("", 72, 45, 67), "Invalid mark")

if _name_ == "_main_":
    unittest.main(verbosity=2)
```

User manual:

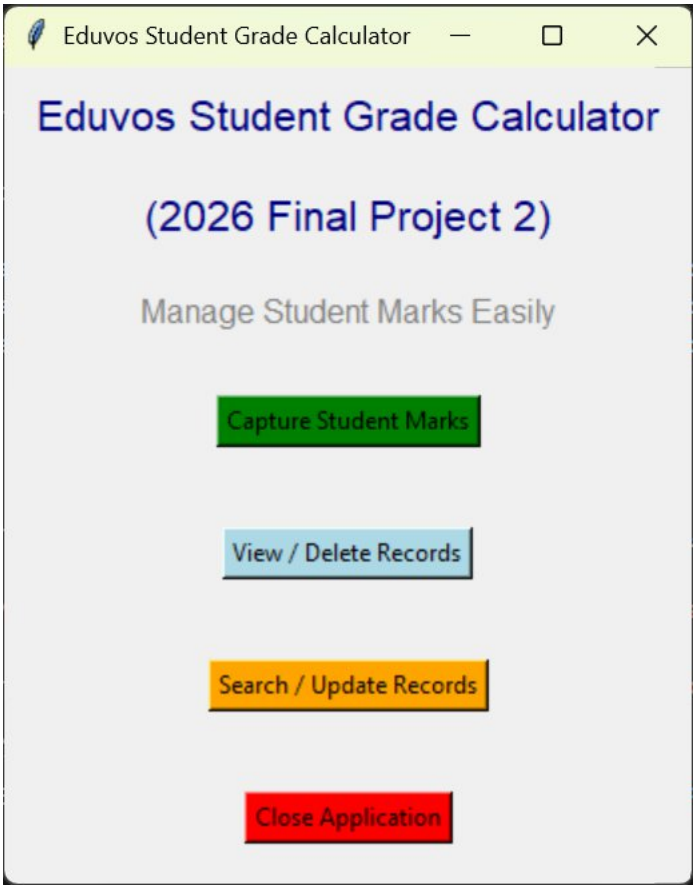
About the System

The Eduvos Student Grade Calculator is a Python application which was designed to manage student marks easily. The application allows users to capture marks, view or delete records, search or update records as well as calculating the marks using the specified grading structure for each module. The application uses a CSV file to store and retrieve data from it. The GUI for the application is made from Tkinter which can be used by the Visual Studio Code IDE.

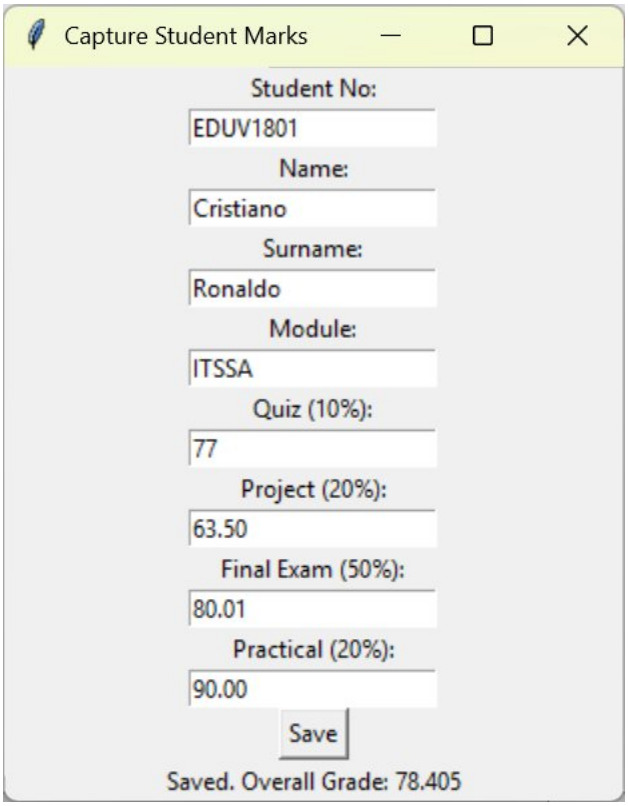
How to Run the Program

The Visual Studio Code IDE can be opened, and the Python file can be loaded onto it. To run the program, there would be a green arrow with the words “Run” next to it which would have to be clicked. Once the program runs, the main window will appear with four (4) buttons which has separate functionalities. The buttons that will appear are the Capture Student Marks, View / Delete Records, Search / Update Records and Close Application. The Close Application terminates the program, and the other buttons has different functionalities as mentioned previously and will be explained further below.

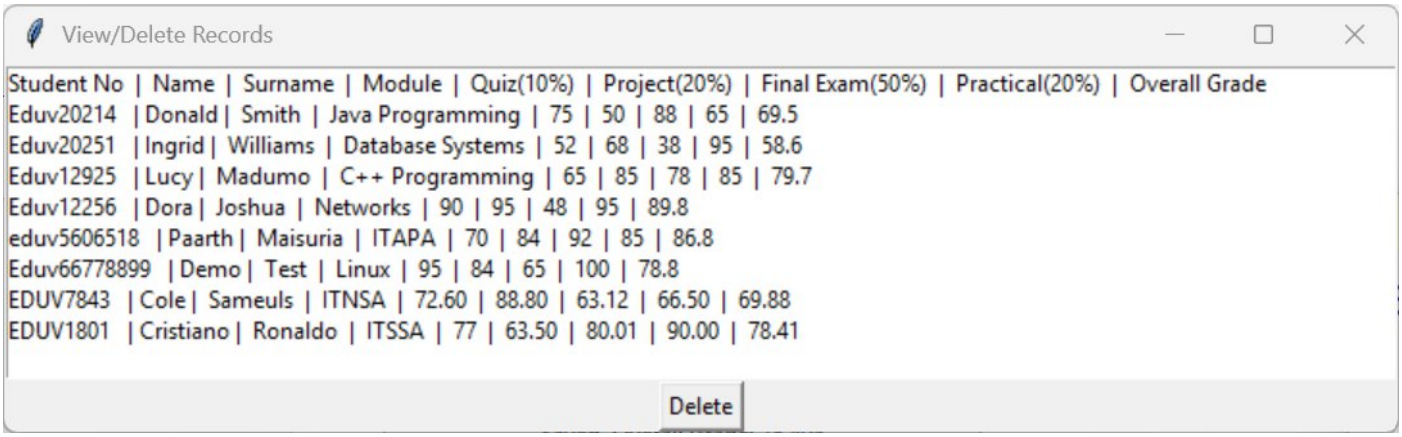
Main Window Functions

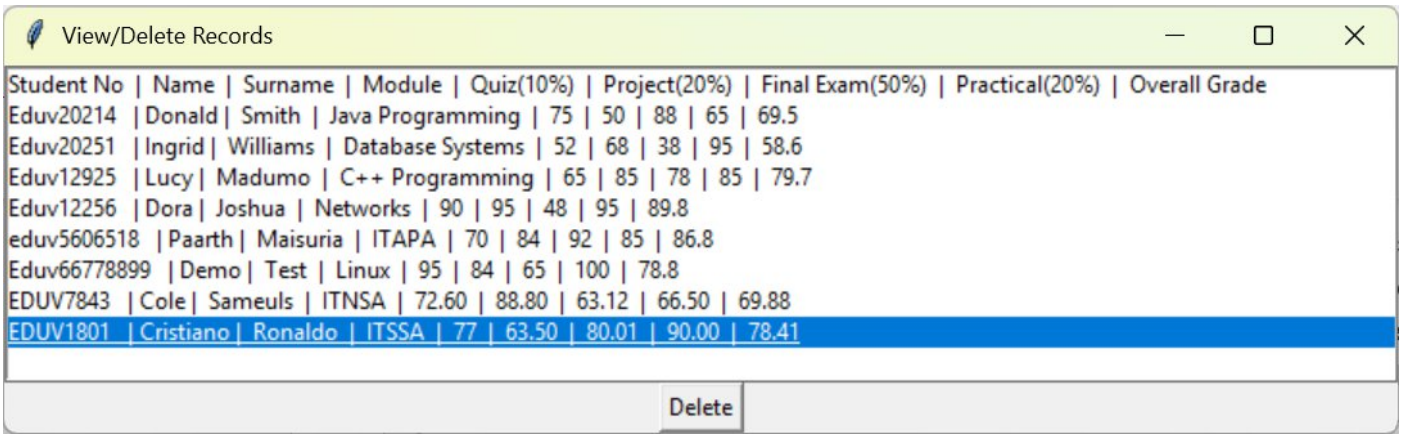


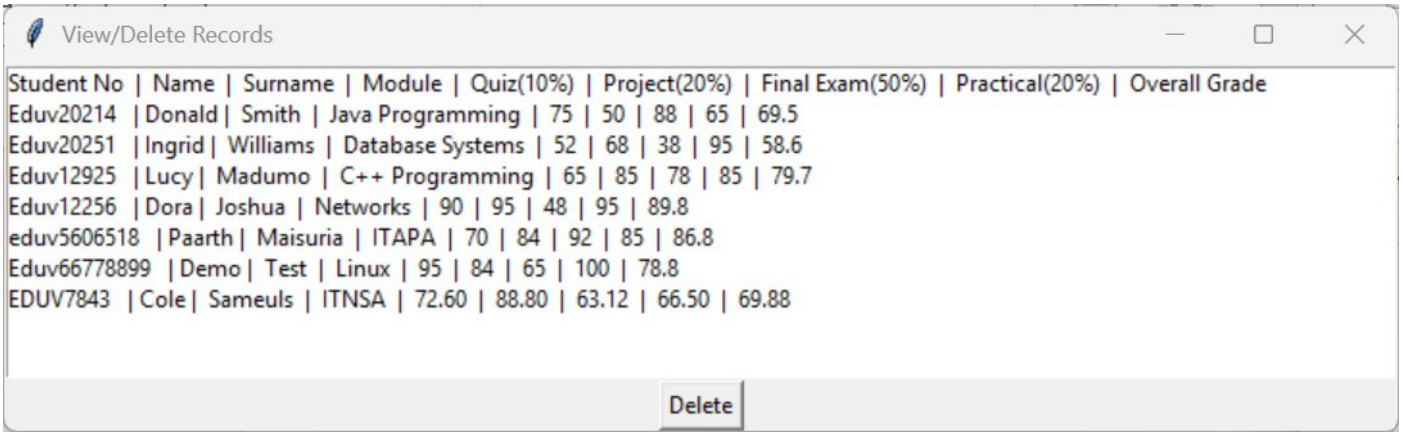
The Capture Student Marks Button – A window is opened which allows the user to input student details and their marks. All fields are required to be populated, and marks should be in the range between 0 and 100. The save button stores the data in the CSV file and also calculates the overall average mark from the inputted marks by the user. The logging system also stores the information about when the student mark was captured into a text file.



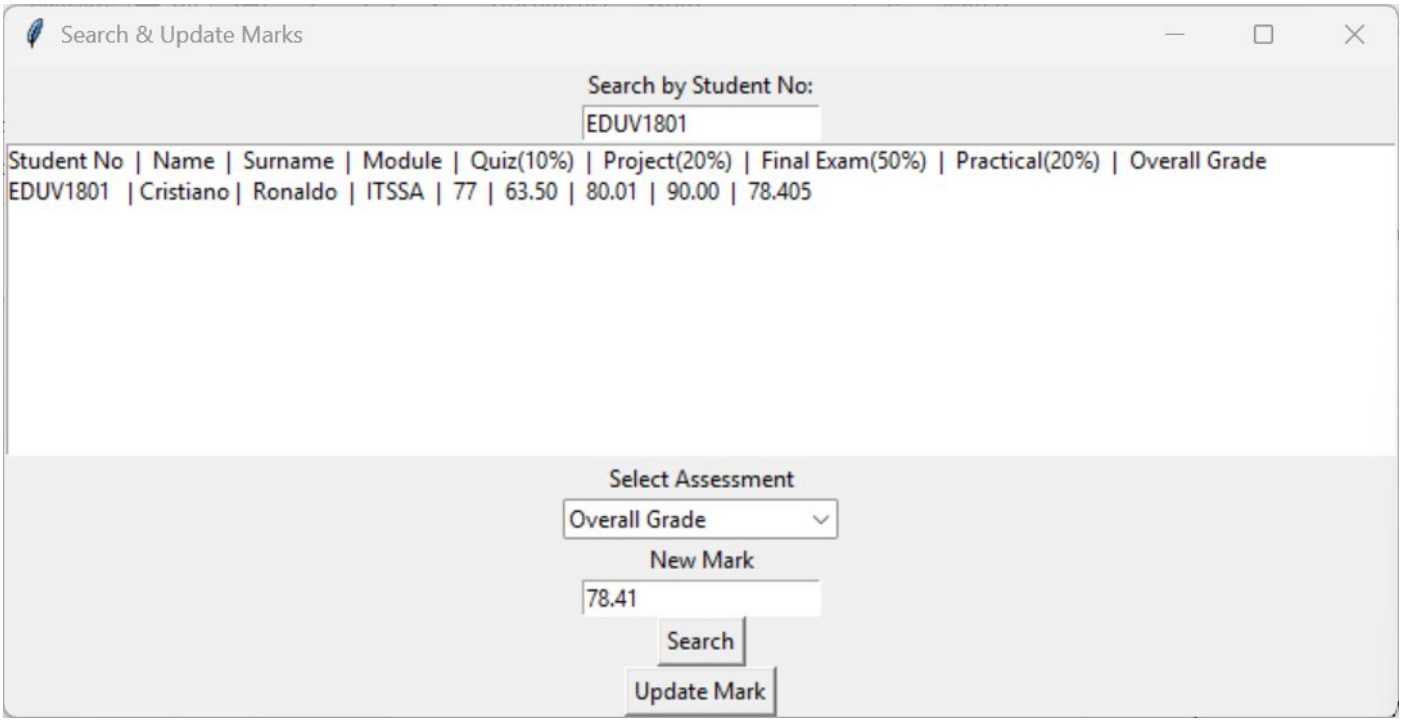
The View / Delete Records Button – A new window is opened and allows the user to view and/or delete records. The records are displayed in a list box with headers. The user can select only one student at a time to delete records, and the system prevents the header row from being deleted. The logging system also stores the information about when the student mark was deleted into a text file.

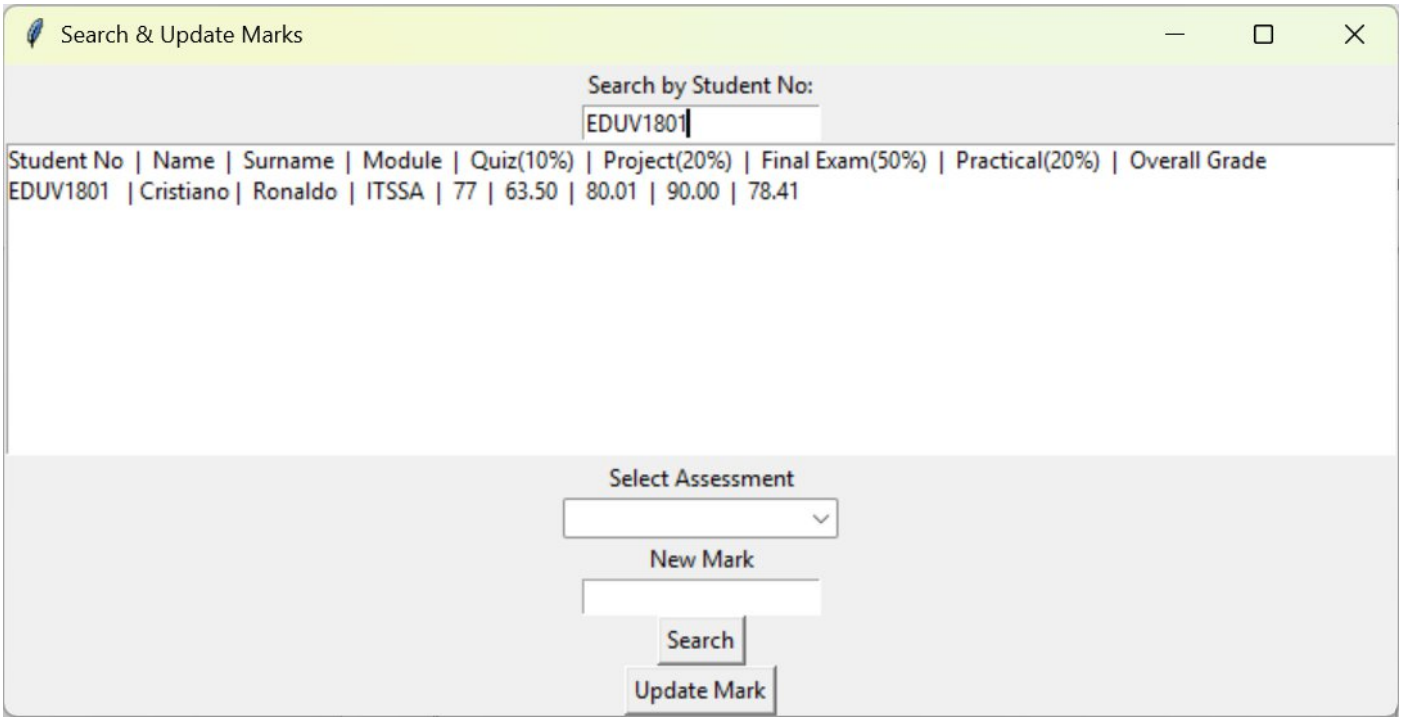






The Search / Update Records Button – Another window is opened which allows the user to search for students by their student numbers. Once the student has been found then the user can update the marks, however, they would only be able to do it individually. The updated marks are stored in the CSV once it has checked that they are valid inputs such as being numeric as well as being in the range between 0 and 100. The logging system also stores the information about when the student mark was updated into a text file.





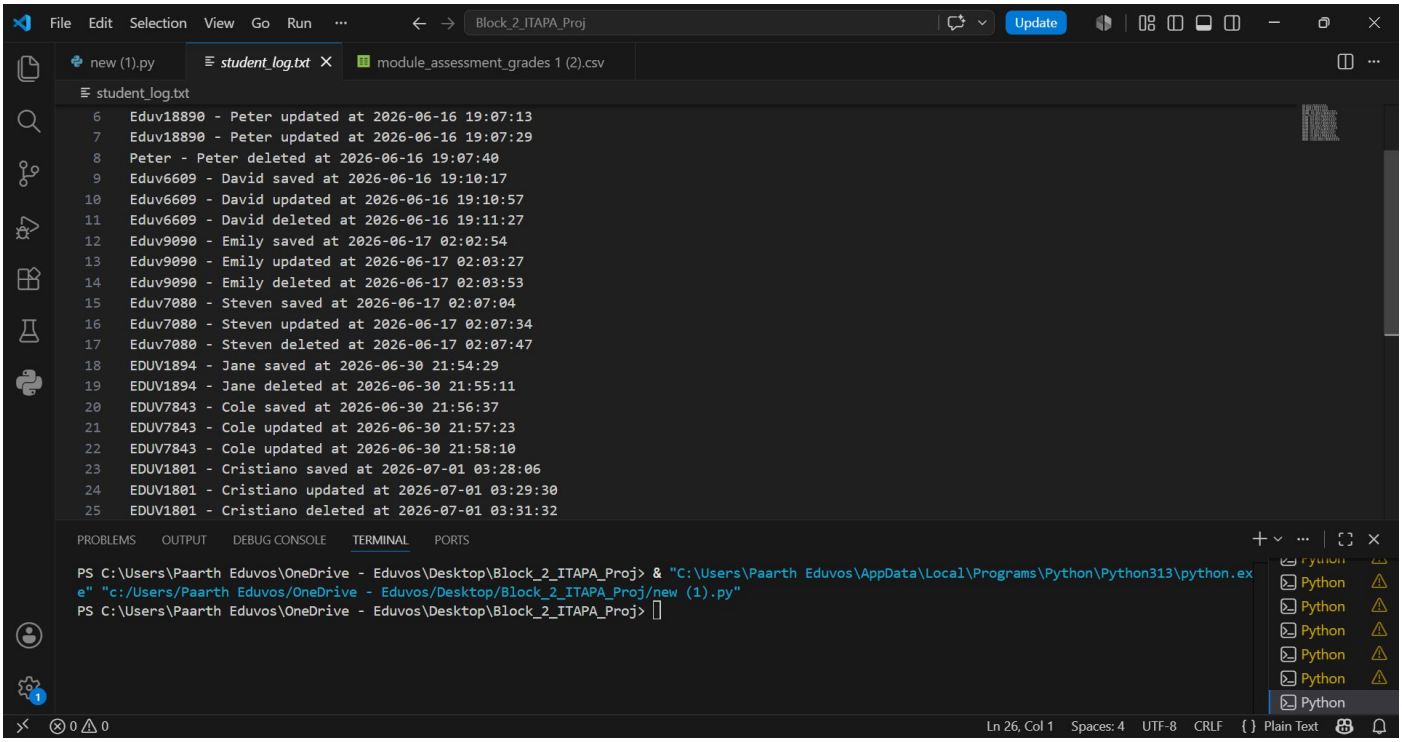
The Close Application Button – The program is terminated safely.

System Components

CSV File Handling – The student records are stored in a CSV file. The functions read_file() and save_student() allows the system to read and write data to the CSV file.

Grade Calculations – The calculate_grade() function checks for the correct input of marks and calculates the grade with the specified instructions from the scenario. It includes quiz – 10%, project – 20%, Exam – 50%, Practical – 20%.

Logging – The log_student() function records the actions taken by the user such as saved, updated, deleted and it's stored in a text file.



Multi-threading – It was used for the logging so that the threads run in the background and the GUI remains functional.

GUI Windows – Tkinter was used to create the separate windows for the different functions in this program.

Final Thoughts About this Program

This manual explains how to run and use the Eduvos Student Grade Calculator program in Visual Studio Code. The program provides a user-friendly interface while also ensuring that data integrity and accountability take place within the system while its being used. Data integrity is ensured through thorough checking of the inputted data from the user and accountability is ensured through logging. Screenshots of each function is provided so that it demonstrates its functionality properly.